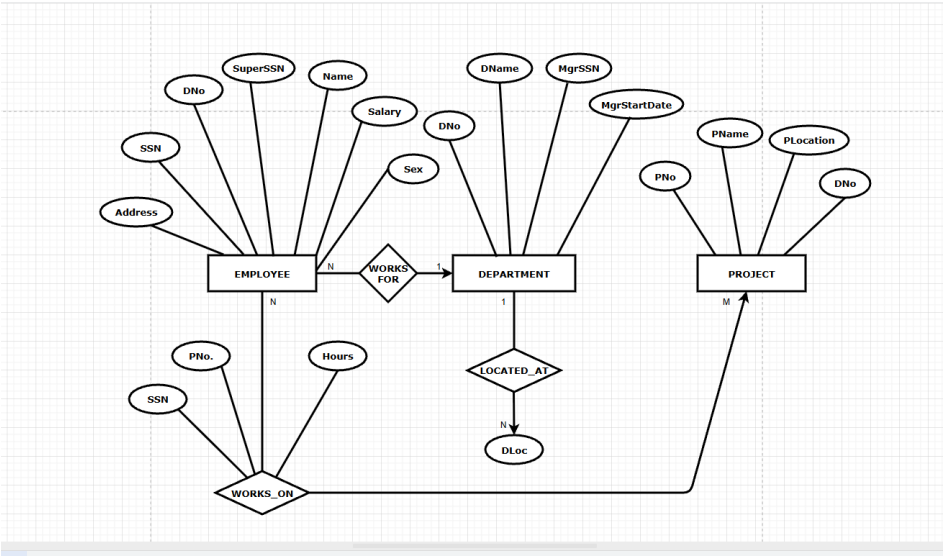
EXPERIMENT-1

1. Consider following databases and draw ER diagram and convert entities and relationships to relation table for a given scenario:

COMPANY DATABASE:
EMPLOYEE (SSN, Name, Address, Sex, Salary, SuperSSN, DNo)
DEPARTMENT (DNo, DName, MgrSSN, MgrStartDate)
DLOCATION (DNo,DLoc)
PROJECT (PNo, PName, PLocation, DNo)
WORKS_ON (SSN, PNo, Hours)



Explanation of ER Diagram – Company Database :

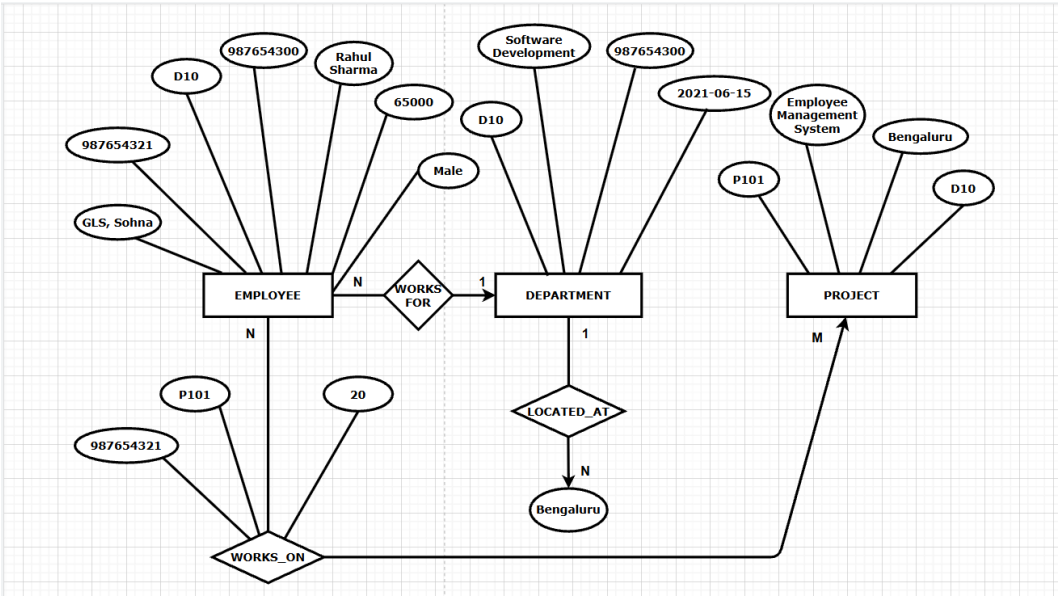
- This ER diagram represents a company database that stores information about employees, departments, locations, and projects.

- The **Employee** entity stores details such as SSN, name, address, sex, salary, supervisor SSN, and department number. Each employee works for a department. This relationship is shown using **WORKS FOR.** Many employees can work for one department, but each employee works for only one department.

- The **Department** entity stores department information such as department number, department name, manager SSN, and manager start date. A department can have multiple locations. This is shown using the **LOCATED AT** relationship between Department and DLocation.

- The **Project** entity stores information about projects such as project number, project name, project location, and department number. Each project is controlled by one department, and one department can control many projects.

- Employees work on different projects. This many-to-many relationship is shown using the **WORKS ON** relationship. The number of hours an employee works on a project is stored as an attribute of this relationship.



Company ER Diagram Explained Using Real Example :

- Consider an employee named **Rahul Sharma** whose SSN is **987654321**. He lives in GLS, Sohna, his salary is 65000, his gender is male, and he works in department **D10**. His supervisor's SSN is **987654300**.

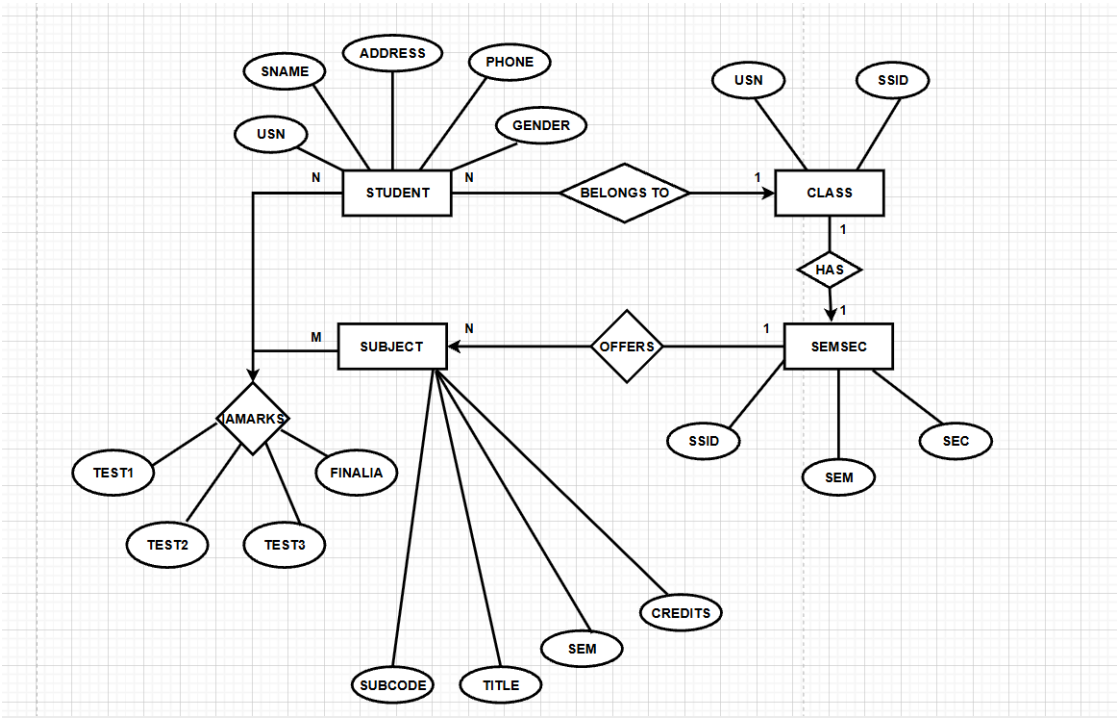
- The department **D10** is called **Software Development**, and its manager SSN is 987654300. The manager started managing the department on 15-06-2021. This department is located in **Bengaluru**.

- One of the projects handled by this department is **Employee Management System** with project number **P101**, and the project is also located in Bengaluru. Rahul Sharma works on this project for **20 hours**. This information is stored using the WORKS ON relationship.

EXPERIMENT-2

2. Consider following databases and draw ER diagram and convert entities and relationships to relation table for a given scenario:

COLLEGE DATABASE:
STUDENT (USN, SName, Address, Phone, Gender)
SEMSEC (SSID, Sem, Sec)
CLASS (USN, SSID)
SUBJECT (Subcode, Title, Sem, Credits)
IAMARKS (USN, Subcode, SSID, Test1, Test2, Test3, FinalIA)



Explanation of ER Diagram – College Database :

- This ER diagram represents a college database that stores information about students, their classes, subjects, semesters, and internal assessment marks.

- In this diagram, the **Student** entity stores basic details of a student such as USN, name, address, phone number, and gender. Every student belongs to a particular class. This relationship is shown using the **BELONGS TO** relationship. Many students can

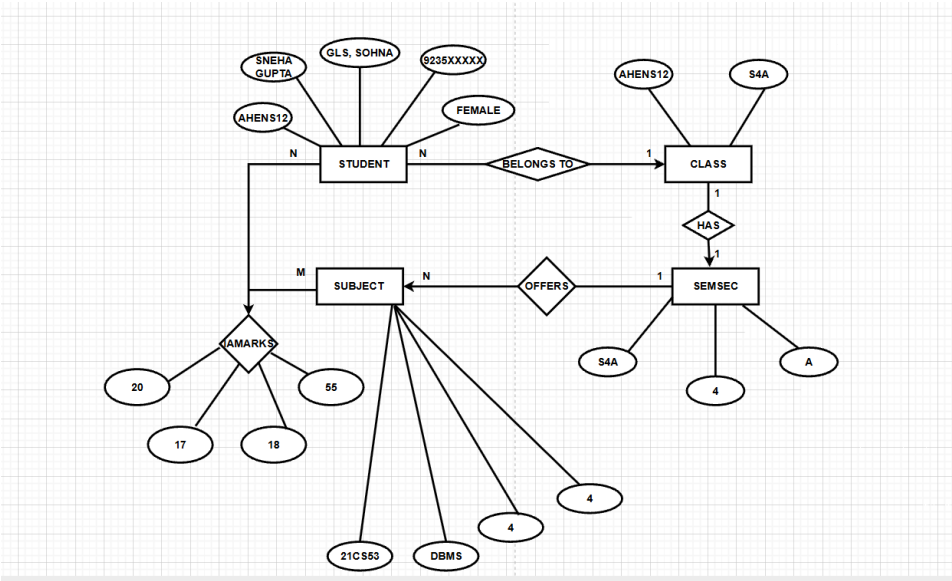
belong to the same class, but a student can belong to only one class.

- The **Class** entity represents a group of students. Each class is associated with exactly one semester and section, which is represented by the **SemSec** entity. The relationship between Class and SemSec is one-to-one, meaning each class has only one semester-section and each semester-section belongs to one class.

- The **SemSec** entity stores information about the semester and section. A semester-section offers multiple subjects. This is shown by the **OFFERS** relationship between SemSec and Subject. One SemSec can offer many subjects, but each subject is offered in one specific semester.

- The **Subject** entity stores subject details such as subject code, title, semester, and credits. Students study multiple subjects, and each subject is studied by many students. This many-to-many relationship is represented using the **IAMARKS** relationship.

- The **IAMARKS** relationship stores the internal assessment marks of students for each subject. It includes Test1, Test2, Test3, and FinalIA. This relationship connects Student and Subject and stores marks for every student-subject combination.



College ER Diagram Explained Using Real Example :

- Consider a student named **Sneha Gupta** whose USN is **AHENS12**. She lives in GLS, Sohna, her phone number is 9235XXXXX, and her gender is female. Sneha belongs to a class named **AHENS12**, which is associated with semester 4 and section A.

- In semester 4, several subjects are offered. One of the subjects is **DBMS** with subject code **21CS53**, semester 4, and 4 credits. Sneha studies DBMS along with other subjects.

- Her internal assessment marks in DBMS are Test1 = 20, Test2 = 17, Test3 = 18, and FinalIA = 55. These marks are stored in the IAMARKS relationship, which connects Sneha to the DBMS subject.

EXPERIMENT-3

Database Schema for a customer-sale scenario:

i. Customer(Cust id : integer, cust_name: string)

ii. Item(item_id: integer, item_name: string, price: integer)

iii. Sale(bill_no:integer,bill_data:date,cust_id: integer, item_id: integer, qty_sold: integer)

iv. For the above schema, perform the following -

```
mysql> create database customer_sale_db;
Query OK, 1 row affected (0.04 sec)

mysql> use customer_sale_db;
Database changed
```

1. Create the tables with the appropriate integrity constraints.

```
mysql> create table customer (
    -> CUST_ID INT PRIMARY KEY,
    -> CUST_NAME VARCHAR(50) NOT NULL);
Query OK, 0 rows affected (0.06 sec)

mysql> SHOW TABLES;
+----------------------------+
| Tables_in_customer_sale_db |
+----------------------------+
| customer                   |
+----------------------------+
1 row in set (0.01 sec)
```

```
mysql> CREATE TABLE ITEM(
    -> ITEM_ID INT PRIMARY KEY,
    -> ITEM_NAME VARCHAR(50) NOT NULL,
    -> PRICE INT NOT NULL CHECK (PRICE>0));
Query OK, 0 rows affected (0.03 sec)

mysql> SHOW TABLES;
+----------------------------+
| Tables_in_customer_sale_db |
+----------------------------+
| customer                   |
| item                       |
+----------------------------+
2 rows in set (0.00 sec)
```

```
mysql> CREATE TABLE SALE (
    -> BILL_NO INT PRIMARY KEY,
    -> BILL_DATE DATE NOT NULL,
    -> CUST_ID INT,
    -> ITEM_ID INT,
    -> QTY_SOLD INT NOT NULL CHECK (QTY_SOLD >0),
    -> FOREIGN KEY (CUST_ID) REFERENCES CUSTOMER(CUST_ID),
    -> FOREIGN KEY (ITEM_ID) REFERENCES ITEM(ITEM_ID));
Query OK, 0 rows affected (0.06 sec)

mysql> SHOW TABLES;
+----------------------------+
| Tables_in_customer_sale_db |
+----------------------------+
| customer                   |
| item                       |
| sale                       |
+----------------------------+
3 rows in set (0.00 sec)
```

2. Insert around 10 records in each of the tables.

```
mysql> INSERT INTO CUSTOMER VALUES
    -> (1,'Rahul'),
    -> (2,'Sneha'),
    -> (3,'Amit'),
    -> (4,'Priya'),
    -> (5,'Karan'),
    -> (6,'Neha'),
    -> (7,'Rohit'),
    -> (8,'Anjali'),
    -> (9,'Vikas'),
    -> (10,'Meena');
Query OK, 10 rows affected (0.03 sec)
Records: 10  Duplicates: 0  Warnings: 0
```

```
mysql> INSERT INTO ITEM VALUES
    -> (101,'Pen',20),
    -> (102,'Notebook',50),
    -> (103,'Bag',500),
    -> (104,'Bottle',150),
    -> (105,'Calculator',300),
    -> (106,'Shoes',1200),
    -> (107,'Watch',800),
    -> (108,'T-Shirt',400),
    -> (109,'Mouse',250),
    -> (110,'Keyboard',600);
Query OK, 10 rows affected (0.02 sec)
Records: 10  Duplicates: 0  Warnings: 0
```

```
mysql> INSERT INTO SALE VALUES
    -> (1,CURDATE(),1,101,5),
    -> (2,CURDATE(),2,103,1),
    -> (3,CURDATE(),5,105,2),
    -> (4,'2026-02-15',3,104,3),
    -> (5,'2026-02-14',4,106,1),
    -> (6,'2026-02-13',6,107,2),
    -> (7,'2026-02-12',7,102,4),
    -> (8,'2026-02-11',8,108,1),
    -> (9,'2026-02-10',9,109,3),
    -> (10,'2026-02-09',10,110,2);
Query OK, 10 rows affected (0.01 sec)
Records: 10  Duplicates: 0  Warnings: 0
```

```
mysql> INSERT INTO CUSTOMER VALUES(11, "ARJUN");
Query OK, 1 row affected (0.02 sec)

mysql> INSERT INTO ITEM VALUES (111, 'LAPTOP STANT', 700);
Query OK, 1 row affected (0.03 sec)

mysql> INSERT INTO SALE VALUES(11,CURDATE(), 11,111,5);
Query OK, 1 row affected (0.01 sec)
```

```
mysql> insert into customer values(12, "DAYA");
Query OK, 1 row affected (0.01 sec)

mysql> insert into item values(112, "touch pen", 660);
Query OK, 1 row affected (0.02 sec)

mysql> insert into sale values (12, CURDATE(), 12, 112, 8);
Query OK, 1 row affected (0.01 sec)
```

3. List all the bills for the current date with the customer names and item numbers.

```
mysql> SELECT S.BILL_NO, C.CUST_NAME , S.ITEM_ID FROM SALE S JOIN CUSTOMER C ON S.CUST_ID = C.CUST_ID WHERE S.BILL_DATE = CURDATE();
+---------+-----------+---------+
| BILL_NO | CUST_NAME | ITEM_ID |
+---------+-----------+---------+
|      11 | ARJUN     |     111 |
|      12 | DAYA      |     112 |
+---------+-----------+---------+
2 rows in set (0.00 sec)
```

4. List the total Bill details with the quantity sold, price of the item and the final amount.

```
mysql> SELECT S.BILL_NO, S.QTY_SOLD, I.PRICE,(S.QTY_SOLD * I.PRICE) AS FINAL_AMOUNT FROM SALE S JOIN ITEM I ON S.ITEM_ID = I.ITEM_ID;
+---------+----------+-------+--------------+
| BILL_NO | QTY_SOLD | PRICE | FINAL_AMOUNT |
+---------+----------+-------+--------------+
|       1 |        5 |    20 |          100 |
|       2 |        1 |   500 |          500 |
|       3 |        2 |   300 |          600 |
|       4 |        3 |   150 |          450 |
|       5 |        1 |  1200 |         1200 |
|       6 |        2 |   800 |         1600 |
|       7 |        4 |    50 |          200 |
|       8 |        1 |   400 |          400 |
|       9 |        3 |   250 |          750 |
|      10 |        2 |   600 |         1200 |
|      11 |        5 |   700 |         3500 |
|      12 |        8 |   660 |         5280 |
+---------+----------+-------+--------------+
12 rows in set (0.01 sec)
```

5. List the details of the customer who have bought a product which has a price>200.

```
mysql> SELECT DISTINCT C.CUST_ID , C.CUST_NAME FROM CUSTOMER C
    -> JOIN SALE S ON C.CUST_ID = S.CUST_ID
    -> JOIN ITEM I ON S.ITEM_ID = I.ITEM_ID
    -> WHERE I.PRICE>200;
+---------+-----------+
| CUST_ID | CUST_NAME |
+---------+-----------+
|       2 | Sneha     |
|       5 | Karan     |
|       4 | Priya     |
|       6 | Neha      |
|       8 | Anjali    |
|       9 | Vikas     |
|      10 | Meena     |
|      11 | ARJUN     |
|      12 | DAYA      |
+---------+-----------+
9 rows in set (0.03 sec)
```

6. Give a count of how many products have been bought by each customer.

```
mysql> SELECT C.CUST_ID, C.CUST_NAME, SUM(S.QTY_SOLD) AS TOTAL_PRODUCTS
    -> FROM CUSTOMER C
    -> JOIN SALE S ON C.CUST_ID = S.CUST_ID
    -> GROUP BY C.CUST_ID, C.CUST_NAME;
+---------+-----------+----------------+
| CUST_ID | CUST_NAME | TOTAL_PRODUCTS |
+---------+-----------+----------------+
|       1 | Rahul     |              5 |
|       2 | Sneha     |              1 |
|       3 | Amit      |              3 |
|       4 | Priya     |              1 |
|       5 | Karan     |              2 |
|       6 | Neha      |              2 |
|       7 | Rohit     |              4 |
|       8 | Anjali    |              1 |
|       9 | Vikas     |              3 |
|      10 | Meena     |              2 |
|      11 | ARJUN     |              5 |
|      12 | DAYA      |              8 |
+---------+-----------+----------------+
12 rows in set (0.03 sec)
```

7. Give a list of products bought by a customer having cust_id as 5.

```
mysql> SELECT I.ITEM_NAME FROM ITEM I JOIN SALE S ON S.ITEM_ID = I.ITEM_ID WHERE CUST_ID=5;
+------------+
| ITEM_NAME  |
+------------+
| Calculator |
+------------+
1 row in set (0.00 sec)
```

8. List the item details which are sold as of today.

```
mysql> SELECT DISTINCT I.* FROM ITEM I JOIN SALE S ON S.ITEM_ID = I.ITEM_ID WHERE S.BILL_DATE =CURDATE();
+---------+--------------+-------+
| ITEM_ID | ITEM_NAME    | PRICE |
+---------+--------------+-------+
|     111 | LAPTOP STANT |   700 |
|     112 | touch pen    |   660 |
+---------+--------------+-------+
2 rows in set (0.01 sec)
```

9. Create a view which lists out the bill_no, bill_date, cust_id, item_id, price, qty_sold, amount.

```
mysql> CREATE VIEW BILL_VIEW AS
    -> SELECT S.BILL_NO,
    -> S.BILL_DATE,
    -> S.CUST_ID,
    -> S.ITEM_ID,
    -> I.PRICE,
    -> S.QTY_SOLD,
    -> (I.PRICE * S.QTY_SOLD) AS AMOUNT
    -> FROM SALE S JOIN ITEM I ON S.ITEM_ID = I.ITEM_ID;
Query OK, 0 rows affected (0.09 sec)

mysql> SELECT * FROM BILL_VIEW;
+---------+------------+---------+---------+-------+----------+--------+
| BILL_NO | BILL_DATE  | CUST_ID | ITEM_ID | PRICE | QTY_SOLD | AMOUNT |
+---------+------------+---------+---------+-------+----------+--------+
|       1 | 2026-02-17 |       1 |     101 |    20 |        5 |    100 |
|       2 | 2026-02-17 |       2 |     103 |   500 |        1 |    500 |
|       3 | 2026-02-17 |       5 |     105 |   300 |        2 |    600 |
|       4 | 2026-02-15 |       3 |     104 |   150 |        3 |    450 |
|       5 | 2026-02-14 |       4 |     106 |  1200 |        1 |   1200 |
|       6 | 2026-02-13 |       6 |     107 |   800 |        2 |   1600 |
|       7 | 2026-02-12 |       7 |     102 |    50 |        4 |    200 |
|       8 | 2026-02-11 |       8 |     108 |   400 |        1 |    400 |
|       9 | 2026-02-10 |       9 |     109 |   250 |        3 |    750 |
|      10 | 2026-02-09 |      10 |     110 |   600 |        2 |   1200 |
|      11 | 2026-02-18 |      11 |     111 |   700 |        5 |   3500 |
|      12 | 2026-02-18 |      12 |     112 |   660 |        8 |   5280 |
+---------+------------+---------+---------+-------+----------+--------+
12 rows in set (0.03 sec)
```

10. Create a view which lists the daily sales date wise for the last one week.

```
mysql> CREATE VIEW WEEKLY_SALES AS
    -> SELECT S.BILL_DATE,
    -> SUM(S.QTY_SOLD * I.PRICE) AS TOTAL_DAILY_SALES
    -> FROM SALE S
    -> JOIN ITEM I ON S.ITEM_ID = I.ITEM_ID
    -> WHERE S.BILL_DATE >= CURDATE() - INTERVAL 7 DAY
    -> GROUP BY S.BILL_DATE;
Query OK, 0 rows affected (0.01 sec)

mysql> SELECT * FROM WEEKLY_SALES;
+------------+--------------------+
| BILL_DATE  | TOTAL_DAILY_SALES  |
+------------+--------------------+
| 2026-02-17 |               1200 |
| 2026-02-15 |                450 |
| 2026-02-14 |               1200 |
| 2026-02-13 |               1600 |
| 2026-02-12 |                200 |
| 2026-02-11 |                400 |
| 2026-02-18 |               8780 |
+------------+--------------------+
7 rows in set (0.02 sec)
```

EXPERIMENT 4.1

Consider the following table:
Table: CLASS

ID	Name
1	Bravo
2	Alex
4	Cheng

1. Insert more values in class table.

```
mysql> create table class (
    -> ID INT PRIMARY KEY,
    -> NAME VARCHAR(30));
Query OK, 0 rows affected (0.10 sec)
```

```
mysql> INSERT INTO CLASS VALUES (11,'Pooja');
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO CLASS VALUES (12,'Rohan');
Query OK, 1 row affected (0.02 sec)

mysql> select * from class;
+----+-------+
| ID | NAME  |
+----+-------+
|  1 | BRAVO |
|  2 | ALEX  |
|  3 | JOHN  |
|  4 | CHENG |
|  5 | RIYA  |
|  6 | KARAN |
|  7 | MEENA |
|  8 | DAVID |
|  9 | Anita |
| 10 | Vikas |
| 11 | Pooja |
| 12 | Rohan |
+----+-------+
12 rows in set (0.00 sec)
```

2. Give the output of the following SQL script:

- INSERT INTO class VALUES (13,’Rahul’)
- COMMIT

```
mysql> start transaction;
Query OK, 0 rows affected (0.02 sec)

mysql> INSERT INTO CLASS VALUES(13,"RAHUL");
Query OK, 1 row affected (0.08 sec)

mysql> COMMIT;
Query OK, 0 rows affected (0.01 sec)
```

- UPDATE class SET name = ‘Abhijeet’ WHERE id= ’13’
- SAVEPOINT A;

```
mysql> UPDATE CLASS SET NAME="ABHIJEET" WHERE ID=13;
Query OK, 1 row affected (0.03 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> select * from class;
+----+----------+
| ID | NAME     |
+----+----------+
|  1 | BRAVO    |
|  2 | ALEX     |
|  3 | JOHN     |
|  4 | CHENG    |
|  5 | RIYA     |
|  6 | KARAN    |
|  7 | MEENA    |
|  8 | DAVID    |
|  9 | Anita    |
| 10 | Vikas    |
| 11 | Pooja    |
| 12 | Rohan    |
| 13 | ABHIJEET |
+----+----------+
13 rows in set (0.01 sec)

mysql> SAVEPOINT A;
Query OK, 0 rows affected (0.01 sec)
```

- INSERT INTO class VALUES (14, ‘Chris’)
- SAVEPOINT B

```
mysql> INSERT INTO CLASS VALUES (14,'CHRIS');
Query OK, 1 row affected (0.00 sec)

mysql> SAVEPOINT B;
Query OK, 0 rows affected (0.00 sec)
```

- INSERT INTO class VALUES (15, ‘Bravo’)
- SAVEPOINT C

```
mysql> INSERT INTO CLASS VALUES (15,'BRAVO');
Query OK, 1 row affected (0.00 sec)

mysql> SAVEPOINT C;
Query OK, 0 rows affected (0.00 sec)
```

- SELECT * FROM class

```
mysql> SELECT * FROM CLASS;
+----+----------+
| ID | NAME     |
+----+----------+
|  1 | BRAVO    |
|  2 | ALEX     |
|  3 | JOHN     |
|  4 | CHENG    |
|  5 | RIYA     |
|  6 | KARAN    |
|  7 | MEENA    |
|  8 | DAVID    |
|  9 | Anita    |
| 10 | Vikas    |
| 11 | Pooja    |
| 12 | Rohan    |
| 13 | ABHIJEET |
| 14 | CHRIS    |
| 15 | BRAVO    |
+----+----------+
15 rows in set (0.00 sec)
```

- ROLLBACK TO B
- SELECT * FROM class

```
mysql> ROLLBACK TO B;
Query OK, 0 rows affected (0.01 sec)

mysql> SELECT * FROM CLASS;
+----+----------+
| ID | NAME     |
+----+----------+
|  1 | BRAVO    |
|  2 | ALEX     |
|  3 | JOHN     |
|  4 | CHENG    |
|  5 | RIYA     |
|  6 | KARAN    |
|  7 | MEENA    |
|  8 | DAVID    |
|  9 | Anita    |
| 10 | Vikas    |
| 11 | Pooja    |
| 12 | Rohan    |
| 13 | ABHIJEET |
| 14 | CHRIS    |
+----+----------+
14 rows in set (0.00 sec)
```

- ROLLBACK TO A

```
mysql> ROLLBACK TO A;
Query OK, 0 rows affected (0.00 sec)
```

```
|  9 | Anita    |
| 10 | Vikas    |
| 11 | Pooja    |
| 12 | Rohan    |
| 13 | ABHIJEET |
+----+----------+
13 rows in set (0.00 sec)
```

EXPERIMENT 4.2

Consider the following table: Table: ACCOUNT_TXN

Acc_ID	Holder_Name	Balance
101	Aman	5000
102	Bhavna	7000
103	Chirag	6000

Perform the following TCL-based SQL script and write the output after each SELECT statement:

```
mysql> Select * from account_txn;
+--------+-------------+---------+
| Acc_ID | Holder_Name | Balance |
+--------+-------------+---------+
|    101 | Aman        |    5000 |
|    102 | Bhavna      |    7000 |
|    103 | Chirag      |    6000 |
+--------+-------------+---------+
3 rows in set (0.02 sec)
```

1. INSERT INTO ACCOUNT_TXN VALUES (104, 'Diya', 4000);
2. SAVEPOINT S1;

```
mysql> INSERT INTO ACCOUNT_TXN VALUES (104, 'Diya', 4000);
Query OK, 1 row affected (0.02 sec)

mysql>
mysql> SAVEPOINT S1;
Query OK, 0 rows affected (0.00 sec)
```

3. UPDATE ACCOUNT_TXN SET Balance = Balance + 1500 WHERE Acc_ID = 101;
4. UPDATE ACCOUNT_TXN SET Balance = Balance - 1000 WHERE Acc_ID = 102;
5. SAVEPOINT S2;

```
mysql> UPDATE ACCOUNT_TXN SET Balance = Balance + 1500 WHERE Acc_ID = 101;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>
mysql> UPDATE ACCOUNT_TXN SET Balance = Balance - 1000 WHERE Acc_ID = 102;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>
mysql> SAVEPOINT S2;
Query OK, 0 rows affected (0.00 sec)
```

6. DELETE FROM ACCOUNT_TXN WHERE Acc_ID = 103;
7. INSERT INTO ACCOUNT_TXN VALUES (105, 'Eshan', 3000);
8. SAVEPOINT S3;

```
mysql> DELETE FROM ACCOUNT_TXN WHERE Acc_ID = 103;
Query OK, 1 row affected (0.00 sec)

mysql>
mysql> INSERT INTO ACCOUNT_TXN VALUES (105, 'Eshan', 3000);
Query OK, 1 row affected (0.00 sec)

mysql>
mysql> SAVEPOINT S3;
Query OK, 0 rows affected (0.00 sec)
```

9. UPDATE ACCOUNT_TXN SET Holder_Name = 'Bhavna Sharma' WHERE Acc_ID = 102;
10. SELECT * FROM ACCOUNT_TXN;

```
mysql> UPDATE ACCOUNT_TXN SET Holder_Name = 'Bhavna Sharma' WHERE Acc_ID = 102;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>
mysql> SELECT * FROM ACCOUNT_TXN;
+--------+--------------+---------+
| Acc_ID | Holder_Name  | Balance |
+--------+--------------+---------+
|    101 | Aman         |    6500 |
|    102 | Bhavna Sharma |    6000 |
|    104 | Diya         |    4000 |
|    105 | Eshan        |    3000 |
+--------+--------------+---------+
4 rows in set (0.00 sec)
```

11. ROLLBACK TO S2;
12. SELECT * FROM ACCOUNT_TXN;

```
mysql> ROLLBACK TO S2;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> SELECT * FROM ACCOUNT_TXN;
+--------+-------------+---------+
| Acc_ID | Holder_Name | Balance |
+--------+-------------+---------+
|    101 | Aman        |    6500 |
|    102 | Bhavna      |    6000 |
|    103 | Chirag      |    6000 |
|    104 | Diya        |    4000 |
+--------+-------------+---------+
4 rows in set (0.00 sec)
```

13. INSERT INTO ACCOUNT_TXN VALUES (106, 'Farah', 4500);
14. SAVEPOINT S4;

```
mysql> INSERT INTO ACCOUNT_TXN VALUES (106, 'Farah', 4500);
Query OK, 1 row affected (0.00 sec)

mysql>
mysql> SAVEPOINT S4;
Query OK, 0 rows affected (0.00 sec)
```

15. UPDATE ACCOUNT_TXN SET Balance = Balance + 500 WHERE Acc_ID IN (101,104);
16. SELECT * FROM ACCOUNT_TXN;

```
mysql> UPDATE ACCOUNT_TXN SET Balance = Balance + 500 WHERE Acc_ID IN (101,104);
Query OK, 2 rows affected (0.00 sec)
Rows matched: 2  Changed: 2  Warnings: 0

mysql>
mysql> SELECT * FROM ACCOUNT_TXN;
+--------+-------------+---------+
| Acc_ID | Holder_Name | Balance |
+--------+-------------+---------+
|    101 | Aman        |    7000 |
|    102 | Bhavna      |    6000 |
|    103 | Chirag      |    6000 |
|    104 | Diya        |    4500 |
|    106 | Farah       |    4500 |
+--------+-------------+---------+
5 rows in set (0.00 sec)
```

17. ROLLBACK TO S4;
18.SELECT * FROM ACCOUNT_TXN;
19. COMMIT;

```
mysql> ROLLBACK TO S4;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> SELECT * FROM ACCOUNT_TXN;
+--------+-------------+---------+
| Acc_ID | Holder_Name | Balance |
+--------+-------------+---------+
|    101 | Aman        |    6500 |
|    102 | Bhavna      |    6000 |
|    103 | Chirag      |    6000 |
|    104 | Diya        |    4000 |
|    106 | Farah       |    4500 |
+--------+-------------+---------+
5 rows in set (0.00 sec)

mysql>
mysql> COMMIT;
Query OK, 0 rows affected (0.01 sec)
```

<u>Student Tasks:</u>

1. Write the output of all 4 SELECT statements in tabular format.
DONE ABOVE!!

2. Identify which rows are permanently saved after COMMIT.

101	Aman	6500
102	Bhavna	6000
103	Chirag	6000
104	Diya	4000
106	Farah	4500

3. Mention which operations were undone by:

a. ROLLBACK TO S2

Delete of Acc_ID 103
Insert of Acc_ID 105
Update name to Bhavna Sharma

b. ROLLBACK TO S4

Balance increase of 500 for Acc_ID 101 and 104

4. Write the final state of the table after COMMIT.

```
mysql> SELECT * FROM ACCOUNT_TXN;
+--------+-------------+---------+
| Acc_ID | Holder_Name | Balance |
+--------+-------------+---------+
|    101 | Aman        |    6500 |
|    102 | Bhavna      |    6000 |
|    103 | Chirag      |    6000 |
|    104 | Diya        |    4000 |
|    106 | Farah       |    4500 |
+--------+-------------+---------+
```

<u>Conceptual Answers :</u>

1. Difference between ROLLBACK and ROLLBACK TO SAVEPOINT
ROLLBACK → Cancels entire transaction.
ROLLBACK TO SAVEPOINT → Cancels changes after that savepoint only.

2. Can we rollback to S3 after ROLLBACK TO S2?
No. Because when we rollback to S2, savepoint S3 is removed automatically.

3. What happens if COMMIT is executed before creating savepoint?
All changes become permanent.
No savepoint can undo them.